

Learn Home Automation with Arduino

Lesson 7: Building a Smart Switch Dashboard

Building a Smart Switch Dashboard – Study Notes

Key Concepts

1. **Client Server Architecture** – How the browser (client) talks to a backend service (e.g., Node RED, Flask, or ESP Home) via HTTP/REST or WebSocket.
2. **HTML /+ /CSS Layout** – Structuring the dashboard with semantic elements (`<section>`, `<article>`, `<button>`) and styling it for a clean, responsive UI.
3. **JavaScript Interaction** – Using `fetch()` (or `axios`) to send commands to the relay endpoints and to poll or subscribe to sensor data.
4. **API Design for IoT Devices** – Typical REST endpoints (`GET /sensors`, `POST /relay/1/on`) and payload formats (JSON).
5. **Security Basics** – Simple measures such as API keys, CORS configuration, and HTTPS for remote access.

Summary

In this lesson you learned how to build a lightweight, browser based dashboard that lets you turn relays on/off and monitor sensor values (temperature, humidity, etc.) from anywhere on the internet. The dashboard consists of three layers:

1. **Presentation layer** – an HTML page styled with CSS to display buttons, status lights, and live sensor readouts.
2. **Logic layer** – JavaScript that issues HTTP requests to a backend API whenever a user clicks a button, and that periodically fetches the latest sensor data to keep the UI up to date.
3. **Backend layer** – a simple REST API (often hosted on the same microcontroller or on a small server) that translates the HTTP calls into GPIO actions (relay control) and reads sensor registers.

By separating concerns, the dashboard remains responsive and easy to extend. You also saw how to protect the API with an API key and enable CORS so the page can be served from a different domain or from a static site host (GitHub Pages, Netlify, etc.).

Important Points

- **HTML structure**

```
```html
<section id="relays">
 <h2>Relays</h2>
 <button data-relay="1" class="relay-btn off">Relay 1 – OFF</button>
 ...
</section>
```

```

<section id="sensors">
 <h2>Sensor Readings</h2>
 <p>Temperature: -- °C</p>
 <p>Humidity: -- %</p>
</section>
```

```

- **CSS tips**

- Use Flexbox/Grid for a responsive layout.
- Visual feedback: `` .on { background:#4caf50; } .off { background:#f44336; } ``

- **JavaScript workflow**

1. Attach click listeners to all `` .relay-btn ``.
2. On click, send ``POST /relay/<id>/<action>`` with the API key in the header.
3. Toggle button class based on the response.
4. Use ``setInterval()`` (e.g., every `/5 /s`) to ``GET /sensors`` and update the DOM.

- **Sample API contract**

```

```http
POST /api/relay/1/on
Headers: X-API-Key: abc123
Response: { "relay":1, "state":"on" }
GET /api/sensors
Response: { "temperature":23.4, "humidity":57 }
```

```

- **Security basics**

- Store the API key in a `` .env `` file on the server; never hard code it in the client.
- Enable CORS only for the domain that serves the dashboard.
- Serve the page over HTTPS (let's encrypt for the server, or use a cloud host that forces TLS).

- **Testing**

- Use browser dev tools ! Network tab to verify request/response payloads.
- Mock the API with tools like **json-server** or **Postman Mock Server** before hardware is ready.

Examples

Example /1 – Turning a Relay On/Off

```

```html
<button id="relay1" class="relay-btn off" data-id="1">Relay /1 – OFF</button>
<script>
const apiKey = 'YOUR_API_KEY';
document.getElementById('relay1').addEventListener('click', async (e) => {
 const btn = e.target;
 const id = btn.dataset.id;
 const action = btn.classList.contains('off') ? 'on' : 'off';
 const resp = await fetch(`/api/relay/${id}/${action}`, {

```

```

 method: 'POST',
 headers: { 'X-API-Key': apiKey }
 });
 const data = await resp.json();
 if (data.state === action) {
 btn.classList.toggle('on');
 btn.classList.toggle('off');
 btn.textContent = `Relay ${id} – ${action.toUpperCase()}`;
 }
});
</script>
...

```

**\*Explanation:** The button's class reflects the current state. Clicking sends a POST request to the appropriate endpoint; on success the UI toggles the class and label.

## Example /2 – Live Sensor Display

```

```html
<div id="sensorBox">
  <p>Temp: <span id="temp">--</span> °C</p>
  <p>Hum: <span id="hum">--</span> %</p>
</div>
<script>
const apiKey = 'YOUR_API_KEY';
async function fetchSensors() {
  const resp = await fetch('/api/sensors', {
    headers: { 'X-API-Key': apiKey }
  });
  const { temperature, humidity } = await resp.json();
  document.getElementById('temp').textContent = temperature.toFixed(1);
  document.getElementById('hum').textContent = humidity.toFixed(0);
}
setInterval(fetchSensors, 5000); // update every 5 /s
fetchSensors(); // initial load
</script>
...

```

***Explanation:** A simple polling loop pulls the latest sensor data and writes it into the page. For real time needs you could replace `setInterval` with a WebSocket listener.

Quick Review

1. ****What HTTP method is typically used to change a relay's state, and why?****
2. ****Name two CSS techniques that help make the dashboard responsive on mobile devices.****
3. ****How can you protect your API from unauthorized use when the dashboard is hosted on a public site?**

**

4. **What are the advantages of using a WebSocket instead of periodic `fetch()` polling for sensor data?

**

5. **If the backend returns `{ "state": "error", "msg": "Invalid API key" }`, how should the front end respond?

**

Further Reading

- **RESTful API Design for IoT** – best practices for naming, versioning, and error handling.
- **WebSocket & MQTT Basics** – low latency alternatives for real time dashboards.
- **CORS and Security Headers** – deeper dive into configuring the server side.
- **Progressive Web Apps (PWA)** – turning the dashboard into an installable offline capable app.
- **Node RED Dashboard** – a visual alternative for rapid prototyping of IoT UIs.

